

PROCESSAMENTO DIGITAL DE IMAGENS

Resumo Completo para Prova de Nível Difícil

Baseado no livro: luizedsilva.github.io/livro-pdi

CAPÍTULO 6 — Compressão de Imagens

6.1 Conceitos Básicos

Compressão busca reduzir o tamanho físico dos dados de uma imagem para otimizar transmissão e armazenamento. O processo envolve duas fases: codificação (compressão) e decodificação (descompressão).

Taxa de Compressão (CR): $CR = n1 / n2$, onde $n1$ = tamanho original e $n2$ = tamanho comprimido.

Redundância de Dados (RD): $RD = 1 - 1/CR$. Ex.: $CR = 4 \rightarrow RD = 0,75 \rightarrow 75\%$ dos dados eram redundantes.

6.2 Tipos de Redundância

- **Espacial:** Pixels vizinhos tendem a ter valores semelhantes. Explorada pelo RLE (Run-Length Encoding): sequências repetidas são substituídas por (valor, quantidade).
- **Estatística (Codificação):** Alguns valores ocorrem mais que outros. Usa-se códigos mais curtos para valores frequentes e mais longos para raros (ex.: Huffman).
- **Psicovisual:** O sistema visual humano não percebe todas as variações de intensidade. Informações pouco perceptíveis podem ser descartadas sem impacto visível (base para compressão com perda).

6.3 Compressão com e sem Perda

- **Sem perda (lossless):** Imagem reconstruída idêntica à original. Ex.: Huffman, LZW, RLE. Usada em imagens médicas, científicas e técnicas.
- **Com perda (lossy):** Permite pequenas distorções em troca de maior taxa de compressão. Ex.: JPEG. Usada em fotos, vídeos e aplicações multimídia.

6.4 Critérios de Fidelidade

Métricas para medir a diferença entre imagem original $f(x,y)$ e reconstruída $f'(x,y)$:

$$MSE = (1/MN) \times \sum [f'(x,y) - f(x,y)]^2$$

$$RMSE = \sqrt{MSE}$$

$$\text{PSNR} = 10 \times \log_{10}(L^2 / \text{MSE}) \text{ [em dB, } L=255 \text{ para 8 bits]}$$

PSNR entre 30 dB e 50 dB é considerado aceitável na prática. Quanto maior o PSNR, mais fiel é a imagem reconstruída.

□ **Prova:** PSNR alto = mais qualidade. MSE alto = mais distorção. São inversamente relacionados.

6.5 Codificação de Huffman

Técnica sem perda que atribui códigos mais curtos a símbolos mais frequentes e códigos mais longos a símbolos raros.

Construção da árvore:

- Determinar a frequência de cada símbolo
- Criar um nó folha para cada símbolo com seu peso (frequência)
- Repetidamente combinar os 2 nós de menor frequência, criando novo nó cujo peso = soma dos dois
- O processo termina quando há apenas um nó (raiz)
- Percorrer da raiz às folhas: ramo esquerdo = bit 0, ramo direito = bit 1

Os códigos gerados são prefixos — nenhum código é prefixo de outro — o que garante decodificação unívoca.

□ **Exercício 4:** Para decodificar: construa a árvore a partir das frequências (A=22, B=17, C=2, D=1). A probabilidade maior gera o menor código. Percorra a sequência de bits na árvore da raiz até as folhas.

6.6 Codificação LZW

Técnica sem perda que não requer conhecimento prévio das probabilidades. Constrói dinamicamente um dicionário de sequências durante a compressão.

- Inicializa o dicionário com todos os símbolos do alfabeto
- Procura a maior sequência presente no dicionário; ao encontrá-la, emite o código e adiciona nova entrada (sequência + próximo símbolo)
- Decodificador reconstrói o mesmo dicionário sem que ele seja transmitido
- Compressão e descompressão em uma única passagem — eficiente para aplicações em tempo real
- Usado em GIF e TIFF

6.7 Padrão JPEG — Etapas Detalhadas

1. Conversão RGB → YCbCr

Separa luminância (Y) de crominância (Cb, Cr). O olho humano é mais sensível a variações de brilho que de cor, portanto Cb e Cr podem ser sub-amostrados.

$$Y = 0,299R + 0,587G + 0,114B$$

$$Cb = -0,1687R - 0,3313G + 0,5B + 128$$

$$Cr = 0,5R - 0,4187G - 0,0813B + 128$$

2. DCT — Transformada Discreta do Cosseno (blocos 8×8)

Converte o domínio espacial para o domínio da frequência. Concentra a energia da imagem em poucos coeficientes de baixa frequência.

$$F(u,v) = (1/4) \cdot C(u) \cdot C(v) \cdot \sum_x \sum_y f(x,y) \cdot \cos[(2x+1)u\pi/16] \cdot \cos[(2y+1)v\pi/16]$$

$C(k) = 1/\sqrt{2}$ se $k=0$; $C(k) = 1$ se $k>0$. O coeficiente $F(0,0)$ é o DC (valor médio do bloco); os demais são AC (variações).

3. Quantização

Principal etapa de perda. Cada coeficiente DCT é dividido por um valor da matriz de quantização $Q(u,v)$ e arredondado.

$$FQ(u,v) = \text{round}[F(u,v) / (k \times Q(u,v))]$$

Coeficientes de alta frequência são divididos por valores maiores → tendem a zero após arredondamento. O fator k controla o grau de compressão.

□ **Prova:** A quantização é a principal fonte de perda no JPEG. Mais agressiva nas altas frequências porque o olho humano é menos sensível a elas.

4. Varredura Zig-Zag

Reorganiza a matriz 8×8 em vetor de 64 elementos, percorrendo diagonalmente do canto superior-esquerdo ao inferior-direito. Coloca coeficientes de baixa frequência no início e os zeros de alta frequência no fim.

5. RLE nos coeficientes AC

Codifica sequências de zeros com pares (r, v): r = quantidade de zeros antes do próximo coeficiente não nulo, v = valor do coeficiente.

$$\text{Ex.: } [15, 0, 0, -3, 0, 0, 0, 2] \rightarrow (0,15), (2,-3), (3,2)$$

- **EOB (End of Block):** Símbolo especial que indica que todos os coeficientes restantes são zero.

6. Codificação Huffman final

Os pares (r,s) dos coeficientes AC e as diferenças DC são codificados com tabelas de Huffman (padrão de luminância ou crominância). Códigos mais curtos para padrões mais frequentes.

DC diferencial: Cada bloco codifica a diferença entre o DC atual e o DC do bloco anterior, pois essas diferenças tendem a ser pequenas.

CAPÍTULO 7 — Morfologia Matemática

7.1 Conceitos Fundamentais

A morfologia matemática analisa imagens com base na forma e estrutura dos objetos. Opera comparando a imagem com um elemento estruturante (sonda).

- **Imagens binárias:** Pixel = 0 (fundo) ou 1 (objeto). Representadas como conjuntos no plano Z^2 .
- **Elemento estruturante (B):** Pequeno conjunto com origem definida. Sua forma, tamanho e orientação determinam o efeito das operações.

7.2 Operações de Conjuntos (base teórica)

Operação	Definição
União $A \cup B$	Pontos em A OU em B
Interseção $A \cap B$	Pontos em A E em B
Complemento A^c	Pontos fora de A (inverte 0↔1)
Simétrico $\check{B}$	Reflexão de B: $\{-p : p \in B\}$
Translação X_u	Deslocamento de X pelo vetor u
Diferença $A \setminus B$	$A \cap B^c$

7.3 Dilatação Binária

Expande as regiões do objeto. Um ponto p pertence ao resultado se B centrado em p intercepta A.

$$\delta_B(I) = I \oplus B = \{p : I \cap B_p \neq \emptyset\} = \bigcup_{p \in I} \{b \in \check{B} : I_p \cap B \neq \emptyset\}$$

Efeitos: preenche lacunas pequenas, conecta componentes próximos, expande bordas externas.

□ **Cálculo prático:** Para cada pixel do resultado: se B (refletido e centrado naquele pixel) tocar pelo menos um pixel '1' da imagem original → resultado = 1.

7.4 Erosão Binária

Contraí as regiões do objeto. Um ponto p pertence ao resultado somente se B centrado em p está completamente contido em A .

$$\varepsilon_B(I) = I \ominus B = \{ p : \check{B}_p \subseteq I \} = \bigcap \{ b \in B \} I_b$$

Efeitos: remove protuberâncias finas, separa objetos conectados por regiões estreitas, elimina ruídos isolados.

□ **Cálculo prático:** Para cada pixel: se B colocado naquele pixel estiver inteiramente sobre pixels '1' $\rightarrow$ resultado = 1. Caso contrário $\rightarrow$ 0.

7.5 Abertura e Fechamento

Abertura = erosão seguida de dilatação (com o mesmo B):

$$\gamma_B(I) = I \circ B = \delta_B(\varepsilon_B(I))$$

Efeito: remove objetos menores que B , suaviza contornos externos, elimina saliências finas.

Fechamento = dilatação seguida de erosão (com o mesmo B):

$$\varphi_B(I) = I \bullet B = \varepsilon_B(\delta_B(I))$$

Efeito: preenche buracos menores que B , conecta regiões próximas, suaviza contornos internos.

7.6 Operações Compostas

- **$d(e(A,B), B)$ — Abertura morfológica:** Erode primeiro, depois dilata. Remove objetos pequenos e ruído, preservando a forma geral.
- **$e(d(A,B), B)$ — Fechamento morfológico:** Dilata primeiro, depois erode. Preenche buracos e conecta objetos próximos.

□ **Exercício 5:** Calcular dilatação: para cada pixel (i,j) da saída, verificar se existe algum pixel '1' em A dentro da vizinhança definida por B centrado em (i,j) . Calcular erosão: exige que TODOS os pixels de B centrado em (i,j) estejam sobre '1' em A .

7.7 Extração de Contornos por Morfologia

O contorno de uma imagem binária pode ser extraído com:

$$\text{Contorno externo} = d(A, B) - A$$

$$\text{Contorno interno} = A - e(A, B)$$

$$\text{Gradiente morfológico} = d(A, B) - e(A, B)$$

□ **Exercício 6:** A extração de contornos usa a diferença entre a imagem dilatada e a original (contorno externo) ou entre a original e a erodida (contorno interno).

7.8 Morfologia em Tons de Cinza

A união e interseção de conjuntos são substituídas por máximo e mínimo locais.

Dilatação em tons de cinza: Atribui a cada pixel o MÁXIMO valor na vizinhança definida por B. Expande regiões claras.

$$(f \oplus b)(x,y) = \max_{\{(s,t) \in b\}} \{ f(x-s, y-t) \}$$

Erosão em tons de cinza: Atribui a cada pixel o MÍNIMO valor na vizinhança definida por B. Expande regiões escuras.

$$(f \ominus b)(x,y) = \min_{\{(s,t) \in b\}} \{ f(x+s, y+t) \}$$

□ **Exercício 7:** Elemento estruturante plano: não usa valores do B nas operações, apenas define a vizinhança. Dilatação → substitui cada pixel pelo maior valor na vizinhança. Erosão → substitui pelo menor.

7.9 Abertura e Fechamento em Tons de Cinza

Abertura: $f \circ b = (f \ominus b) \oplus b \rightarrow$ remove picos claros

Fechamento: $f \bullet b = (f \oplus b) \ominus b \rightarrow$ preenche vales escuros

- **Top-hat (white top-hat):** $f - (f \circ b) \rightarrow$ evidencia objetos claros menores que B.
- **Bottom-hat (black top-hat):** $(f \bullet b) - f \rightarrow$ evidencia objetos escuros menores que B.

7.10 Propriedades das Operações Morfológicas

- **Idempotência:** Abertura e fechamento aplicados duas vezes produzem o mesmo resultado.
- **Dualidade:** Erosão e dilatação são duais; abertura e fechamento também.
- **Invariância por translação:** O resultado não depende da posição absoluta dos objetos.

CAPÍTULO 8 — Segmentação de Imagens

8.1 Conceito e Propriedades Formais

Segmentação divide a imagem em regiões R_i distintas e homogêneas. Condições formais:

- União de todas as regiões = imagem completa: $\bigcup R_i = I$
- Regiões disjuntas: $R_i \cap R_j = \emptyset$ para $i \neq j$
- Cada região satisfaz critério de homogeneidade
- Regiões adjacentes NÃO são homogêneas pelo mesmo critério

8.2 Limiarização

Técnica mais simples. Define limiar T para binarizar a imagem:

$$g(x,y) = 1 \text{ se } I(x,y) \geq T ; 0 \text{ caso contrário}$$

- **Global:** Mesmo T para toda a imagem. Funciona bem para histogramas bimodais.
- **Adaptativa:** T varia localmente com base em características da vizinhança (média, mediana).
- **Método de Otsu:** Calcula T automaticamente maximizando a variância entre as duas classes (objeto e fundo).

8.3 Watershed — Algoritmo Completo

Interpreta a imagem como superfície topográfica: intensidade = altitude. Regiões de baixa intensidade = vales (bacias). Regiões de alta = divisores de água.

O algoritmo simula inundação a partir dos mínimos locais. Quando águas de diferentes bacias se encontram, forma-se a linha watershed (borda).

Problema: Supersegmentação causada por múltiplos mínimos locais (ruído). **Solução:** usar marcadores.

Watershed por Marcadores — Passo a Passo

- Cada marcador recebe um rótulo distinto
- Vizinhos dos marcadores são inseridos em fila de prioridade (ordenada por nível de cinza)
- Para cada pixel x extraído da fila:
 - → Se todos os vizinhos já rotulados têm o mesmo rótulo: x recebe esse rótulo
 - → Se vizinhos têm rótulos DISTINTOS: x é marcado como watershed (fronteira)
- Vizinhos ainda não rotulados de x são inseridos na fila
- Repete até fila vazia

□ **Exercício 9:** Watershed é um método de segmentação por morfologia. É implementado com fila de prioridade baseada nos níveis de cinza, simulando inundação a partir de marcadores. Aplica-se tipicamente sobre a imagem de gradiente morfológico.

8.4 Gradiente Morfológico

Usado como entrada para o Watershed. Evidencia bordas:

Gradiente = $d(A, B) - e(A, B)$ [dilatação menos erosão]

CAPÍTULO 9 — Representação e Descrição de Imagens

9.1 Código de Cadeia (Chain Code)

Representa o contorno de um objeto como sequência de direções discretas. Transforma a borda 2D em uma sequência 1D de números inteiros.

Direções — 8-direcional (mais comum)

Código	Direção (desloc. linha, coluna)
0	(0, +1) → leste
1	(-1, +1) → nordeste
2	(-1, 0) → norte
3	(-1, -1) → noroeste
4	(0, -1) → oeste
5	(+1, -1) → sudoeste
6	(+1, 0) → sul
7	(+1, +1) → sudeste

4-direcional

Usa apenas 0 (leste), 1 (norte), 2 (oeste), 3 (sul). Menos preciso, mas mais compacto.

Normalização do Código

O código depende do ponto inicial. Para torná-lo invariante à escolha do ponto de partida: considerar todas as rotações cíclicas da sequência e selecionar a que representa o menor valor inteiro (ordem lexicográfica).

Propriedades do Código de Cadeia

- **Invariância à translação:** O código não muda com a posição do objeto (apenas depende da sequência de movimentos).
- **Invariância à rotação:** Pode ser obtida calculando a primeira diferença da cadeia (diferença módulo 4 ou 8 entre elementos consecutivos).
- **Comprimento do contorno:** Em 4-direcional = número de elementos; em 8-direcional = conta passos horizontais/verticais como 1 e diagonais como $\sqrt{2}$.

Codificação em bits (Exercício 11)

4 direções = 2 bits por passo (0=00, 1=01, 2=10, 3=11).

8 direções = 3 bits por passo (0=000, 1=001, ..., 7=111).

$$\text{Número de bytes} = \lceil (n_{\text{passos}} \times \text{bits_por_passo}) / 8 \rceil$$

□ **Exercício 10 e 11:** Para encontrar o menor valor base 4 ou base 8: normalizar o código (rotação que gera o menor valor) e interpretar a sequência como número na base correspondente.

9.2 Código de Cadeia e Formato de Imagem Simplificado (Ex. 12)

A borda de uma imagem pode ser representada de forma compacta usando:

- Ponto inicial (linha, coluna) do contorno
- Número de pontos
- Sequência do código de cadeia (em bits ou hex)

Este formato armazena apenas os contornos, sem os pixels internos, reduzindo drasticamente o tamanho do arquivo para imagens com objetos bem definidos. A decodificação reconstrói os pontos do contorno percorrendo as direções a partir do ponto inicial.

□ **Exercício 12:** A codificação de cadeia dos contornos serve como base para um formato de imagem que representa apenas as bordas. Útil para reconhecimento de padrões, compressão de formas e transmissão eficiente de imagens binárias ou de contornos.

9.3 Descritores de Região

- **Área:** Número de pixels pertencentes à região.
- **Perímetro:** Número de pixels de borda.
- **Compacidade:** P^2 / A (quanto menor, mais circular).
- **Momentos de ordem (p,q):** $m_{pq} = \sum_x \sum_y x^p \cdot y^q \cdot f(x,y)$
- **Momentos invariantes de Hu:** Descritores invariantes à translação, rotação e escala.

9.4 Esqueletização

Reduz a forma a um eixo central (esqueleto) com espessura de 1 pixel, preservando topologia e conectividade. Implementada por afinamento morfológico iterativo.

TABELA RESUMO — Exercícios × Conteúdo

Ex.	Tema	Conceito-chave
2	JPEG	DCT → Quantização → Zig-Zag → RLE → Huffman

3	Compressão com perda	Redundância psicovisual; PSNR como métrica de qualidade
4	Huffman	Árvore por frequência; decodificar percorrendo a árvore
5a	Dilatação binária	$d(A,B)$: resultado=1 se B intercepta A naquele pixel
5b	Erosão binária	$e(A,B)$: resultado=1 somente se $B \subseteq A$ naquele pixel
5c	Abertura	$d(e(A,B),B)$: remove objetos menores que B
5d	Fechamento	$e(d(A,B),B)$: preenche buracos menores que B
6	Contorno morfológico	Contorno = $A - e(A,B)$ ou $d(A,B) - A$
7	Morfologia tons de cinza	Dilatação→MAX; Erosão→MIN na vizinhança de B
8	Borda morfológica	Extrair borda da região ≥ 10 via gradiente morfológico
9	Watershed	Fila de prioridade; inundação a partir de marcadores
10	Código de cadeia	Sequências de direções 4 e 8; menor valor por normalização
11	Código em bytes	4 dir=2 bits/passo; 8 dir=3 bits/passo; bytes= $\lceil n \times \text{bits} / 8 \rceil$
12	Formato de imagem	Ponto inicial + número de pontos + cadeia → formato compacto

Baseado exclusivamente em: luizedsilva.github.io/livro-pdi | UNIFAL-MG — PDI